

Exercise 1: Spring Data JPA - Quick Example (Country Retrieval)

Scenario

This exercise demonstrates how to build a Spring Boot application using **Spring Data JPA and Hibernate** to retrieve data from a MySQL database table (country). The application follows a layered architecture using Entity, Repository, and Service classes.

Objective

The objective of this exercise is to understand how to:

- Configure Spring Boot with Spring Data JPA
 - Connect Spring Boot application with MySQL database (XAMPP)
 - Create and map entity classes using JPA annotations
 - Use JpaRepository to perform database operations
 - Implement service layer with transaction support
 - Retrieve and display data from database using Spring Boot application
-

1. Understanding Spring Data JPA

Spring Data JPA simplifies database operations by:

- Eliminating boilerplate JDBC code
- Providing built-in CRUD operations
- Supporting repository-based data access
- Integrating Hibernate as ORM provider

It helps developers focus on business logic instead of SQL queries.

2. Dependencies

The following dependencies are used in pom.xml:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-data-jpa</artifactId>  
</dependency>
```

```
<dependency>

    <groupId>com.mysql</groupId>

    <artifactId>mysql-connector-j</artifactId>

    <scope>runtime</scope>

</dependency>
```

```
<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-devtools</artifactId>

</dependency>
```

3. Database Setup (MySQL - XAMPP)

Database Name:

ormlearn

Table Creation:

```
create table country(
    co_code varchar(2) primary key,
    co_name varchar(50)
);
```

Sample Data:

```
insert into country values ('IN', 'India');
```

```
insert into country values ('US', 'United States of America');
```

4. Implementation

4.1 Entity Class (Country.java)

This class maps Java object to database table.

```
@Entity
```

```
@Table(name = "country")
```

Fields:

- code → co_code (Primary Key)

- name → co_name
-

4.2 Repository Layer

public interface CountryRepository extends JpaRepository<Country, String>

This interface provides built-in methods such as:

- findAll()
 - findById()
 - save()
 - delete()
-

4.3 Service Layer

The service layer contains business logic and uses repository methods.

@Transactional

```
public List<Country> getAllCountries() {  
    return countryRepository.findAll();  
}
```

4.4 Main Application (Execution Flow)

The application starts from main() method:

Steps:

1. Spring Boot application starts
 2. ApplicationContext is created
 3. CountryService bean is retrieved
 4. getAllCountries() method is executed
 5. Data is fetched from MySQL database
-

5. Output

When the application runs successfully, the following output is displayed:

Inside main

Start

Repository called

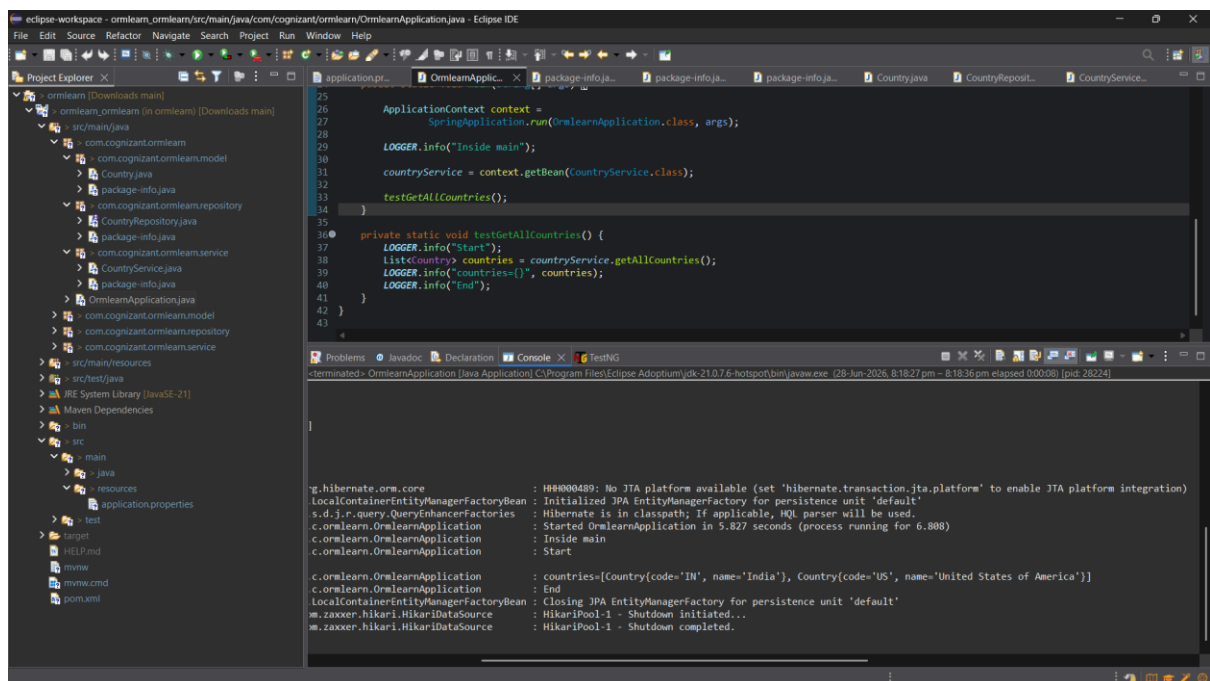
```
countries=[Country{code='IN', name='India'}, Country{code='US', name='United States of America'}]
```

End

6. Conclusion

This exercise demonstrates the successful implementation of **Spring Boot with Spring Data JPA** to perform database operations. It shows how a layered architecture (Entity–Repository–Service) simplifies database access and improves code maintainability. The application successfully retrieves data from a MySQL database using Hibernate ORM.

OUTPUT SCREENSHOT



The screenshot displays the Eclipse IDE interface. The Project Explorer on the left shows the project structure for 'ormlearn'. The main editor window displays the 'OrmlearnApplication.java' file, which contains the following code:

```
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44
```

```
ApplicationContext context =  
    SpringApplication.run(OrmlearnApplication.class, args);  
  
LOGGER.info("Inside main");  
  
countryService = context.getBean(CountryService.class);  
testGetAllCountries();  
  
private static void testGetAllCountries() {  
    37  
    38  
    39  
    40  
    41  
    42  
    43  
    44
```

```
LOGGER.info("Start");  
List<Country> countries = countryService.getAllCountries();  
LOGGER.info("countries={}", countries);  
LOGGER.info("End");  
}
```

The Console window at the bottom shows the output of the application:

```
<terminated> OrmlearnApplication [Java Application] C:\Program Files\Eclipse Adoptium\jdk-21.0.7-hotspot\bin\java.exe (28-Jun-2025, 8:18:27 pm - 8:18:36 pm elapsed 0:00:08) [pid: 28224]  
  
g.hibernate.orm.core : 184800489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)  
LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'  
s.d.j.r.query.QueryEnhancerFactories : Hibernate is in classpath; If applicable, HQL parser will be used.  
c.Ormlearn.OrmlearnApplication : Started OrmlearnApplication in 5.827 seconds (process running for 6.808)  
c.Ormlearn.OrmlearnApplication : Inside main  
c.Ormlearn.OrmlearnApplication : Start  
  
c.Ormlearn.OrmlearnApplication : countries=[Country{code='IN', name='India'}, Country{code='US', name='United States of America'}]  
c.Ormlearn.OrmlearnApplication : End  
LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'  
m.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown initiated...  
m.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown completed.
```